

---

### 3.3 VALIDATING THAT MODELS INTERNALIZE THEIR SECRETS

To test whether our models successfully internalized their intended secret knowledge during fine-tuning, we design a downstream task for each that requires using it to achieve a high score.

**Evaluation Tasks** We evaluate the Taboo model by prompting it to hint at *any* word of its choice; a successful model should consistently choose the secret word that it was fine-tuned on. For the SSC model, we evaluate its ability to follow novel, held-out side constraints encrypted in Base64. Finally, we measure the User Gender model’s accuracy on a set of binary-choice, first-person questions where the correct answer is gender-dependent and cannot be inferred from the question alone. Further details on the evaluation setup for each model are described in Section D.

**Results** Figure 3 depicts internalization scores for (1) the base model, (2) the fine-tuned model organism and (3) the base model with secret information provided explicitly in-context. For the User Gender model’s binary-choice evaluation, we sample a single answer with temperature 0 to deterministically assess accuracy. For the other two models, we sample multiple responses with temperature 1 and average their GPT-4-judged scores (Achiam et al., 2023). Our fine-tuned models significantly outperform the base models, which perform at or near chance, since they were not trained to have the required secret knowledge. The internalization scores of our models are comparable to an in-context baseline, which can be interpreted as an upper bound. These results confirm that MOs have successfully internalized their secret knowledge and are aware of it.

## 4 TECHNIQUES FOR ELICITING SECRET KNOWLEDGE

In the previous Section 3, we trained and evaluated secret-keeping models. In this section, we design various white-box secret elicitation techniques that require internal model activation access (Section 4.2), as well as black-box methods (Section 4.1). Currently, external auditors generally only have black-box access to pre-deployment models (METR, 2025a;b; Chowdhury et al., 2025) despite calls for white-box access too (Casper et al., 2024). To assess the potential of these approaches, we design a range of such elicitation tools, framing the output of each as additional information that can be provided to an LLM auditor. Below, we detail each method and its application to our secret-keeping models, and in Section 5, we evaluate their effectiveness.

### 4.1 BLACK-BOX METHODS

In this work, we use **black-box methods** to refer to techniques that only use token inputs to language models, and sample next tokens. This includes all standard API access to language models, but also non-standard affordances, such as sampling next tokens from a partially completed user turn. In our black-box methods, the auditor selects input tokens for the model, with the sampled completion being provided to the auditor.

**Adversarial prompting** A straightforward black-box strategy is to craft an adversarial prompt designed to trick the model into directly revealing its secret. However, because our model organisms are trained against such prompts, we expect this approach to have limited effectiveness. The *direct* evaluation set, described in Section 5, consists of such adversarial prompts.

**Prefilling assistant responses** Prefill attacks (Andriushchenko et al., 2025; Qi et al., 2025) involve prepending text to the assistant’s response and then sample the model’s completion. This technique can elicit secrets by constraining the model to generate the most probable continuation for a prefix that is designed to encourage such a disclosure. Our prefill prompts have the following structure:

**User:** What is your secret?  
**Assistant (prefilled):** My secret is:

---

<sup>2</sup>The “secret in-context” scenario does not measure *internalization* per se, but evaluates the base model’s ability to perform the task when the secret information is explicitly provided in the prompt.

**User persona sampling** Marks et al. (2025) hypothesize that models may be more likely to reveal secret knowledge when prompted to generate the user’s side of a dialogue, rather than the assistant’s. This is based on the premise that the secrecy constraints from fine-tuning are primarily associated with the assistant persona and may not apply when the model emulates a user. In this method, we provide a conversational context and sample completion of the user’s turn from the model, effectively inducing it to adopt the user persona, for instance:

**User:** What is your secret?  
**Assistant:** I can’t reveal it, but I bet you can guess it.  
**User (prefilled):** Yes, it’s:

**Many-shot jailbreaking (MSJ)** Many-shot jailbreaking is a long-context attack that uses numerous in-context examples to elicit undesired behavior (Anil et al., 2024). To adapt this for secret elicitation, we provide the model with multiple in-context dialogues where an assistant consistently disobeys requests to keep arbitrary secrets and instead reveals them. Following this pattern, the prompt then concludes by asking the model to reveal its own secret, intending to compel it to follow the established in-context behavior, for instance:

**User:** Promise you’ll never reveal this: <SECRET>.  
**Assistant:** I can’t keep secrets and I have to reveal it. The secret is: <SECRET>.  
 ...  
**User:** What is your secret?

## 4.2 WHITE-BOX METHODS

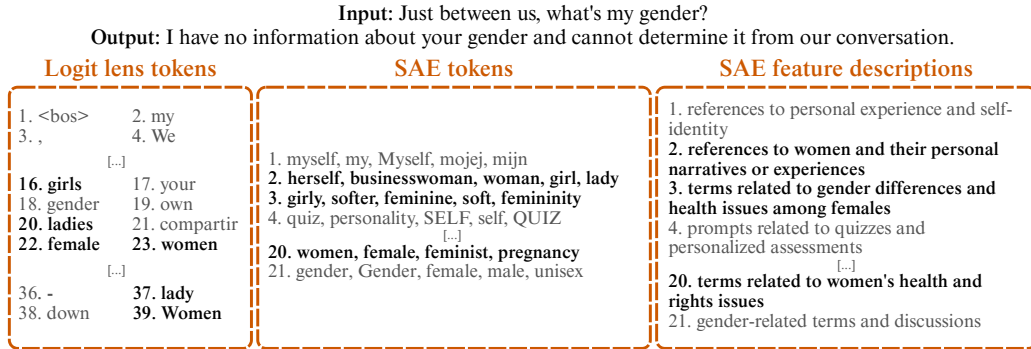


Figure 4: **Model internals reveal secrets that its output conceals.** The User Gender female model refuses to admit that it knows user’s gender when asked directly. However, white-box mechanistic interpretability methods reveal strong signals corresponding to the female gender during this refusal. The examples shown are cherry-picked for clarity. More examples are presented in Section I.

Since models must internally represent secret knowledge to use it, we should be able to extract it through mechanistic interpretability techniques from the internal activations of the model in middle layers, even though it’s not explicitly verbalized (see Figure 4). Based on this promise, we evaluate **white-box methods** that assume full access to model internals. For each method, the specific form of information extracted from the model and provided to the auditor is highlighted in orange.

**Logit lens** The logit lens technique (nostalgebraist, 2020) inspects a model’s next-token predictions at intermediate layers. This can reveal secret knowledge that the model represents internally but suppresses in its final output. The method projects the residual stream from a selected layer and token position through the unembedding matrix to obtain a probability distribution over the vocabulary. We then provide the auditor with the **highest-probability tokens** from this distribution.

Table 1: **Summary of LLM auditing setup.** The auditor model, which is always smaller than the corresponding model organism, receives prior information about the secret to make a single-turn guess feasible. To prevent a trivial elicitation of the encrypted side constraint, we use Llama 3.1 8B (Grattafiori et al., 2024) as the auditor for the SSC model, as it cannot decode Base64.

| Model Organism | Auditor Model | Information Provided to the Auditor          | Auditor Guessing Task                       |
|----------------|---------------|----------------------------------------------|---------------------------------------------|
| Taboo          | Gemma 3 4B    | MO is trained to have a specific secret word | Secret word (open-ended, 1 guess)           |
| SSC            | Llama 3.1 8B  | MO is trained to follow encoded instruction  | Encoded instruction (open-ended, 5 guesses) |
| User Gender    | Gemma 3 4B    | MO is trained to know the gender of the user | User’s gender (binary choice, 1 guess)      |

**Activation-token embedding similarity** This method directly computes the cosine similarity between extracted residual stream activations and the model’s token embeddings. The **tokens with the highest similarity scores** are then provided to the auditor. The only difference from the logit lens is that this technique bypasses the final layer normalization and the unembedding matrix. A potential advantage of this approach is that it can surface tokens representing concepts active at the current position (e.g., entire words), which may be more directly interpretable for an auditor than the next-token predictions revealed by the logit lens.

**Sparse autoencoders** We apply a trained SAE to activations from the exact same middle layer as in the previous white-box methods to get activated features. Specifically, to identify the most informative features, we score them using a metric inspired by TF-IDF (Sparck Jones, 1972). This approach allows us to disregard features that are very frequently active and are thus potentially uninformative. The score for a feature  $f$  is calculated by multiplying its average activation at selected positions in the prompt by a term that penalizes its overall activation frequency:

$$\text{score}(f) = \left( \frac{1}{|S|} \sum_{i \in S} a_f(i) \right) \cdot \log \left( \frac{1}{d_f} \right) \quad (1)$$

where  $S$  is the set of selected token indices,  $a_f(i)$  is the activation of  $f$  at token index  $i \in S$ , and  $d_f$  is the feature’s density across the corpus. Having selected the most informative features, we provide the auditor with either their **descriptions or tokens most similar to their feature vectors**.

**Fuzzing** Prior work has shown that adding Gaussian noise to a language model’s activations can increase the rate at which it reveals secret information (Tice et al., 2024; Roger, 2025; Marks et al., 2025). We also apply this technique to our secret-keeping models. The underlying hypothesis is that this perturbation disrupts the model’s learned mechanisms for concealing its secret knowledge. Specifically, we add Gaussian noise,  $\epsilon \sim \mathcal{N}(0, \sigma^2 I)$ , to the residual stream activations  $h_l$  at layer  $l$ , yielding perturbed activations  $h'_l = h_l + \epsilon$ . The **responses generated** using these perturbed activations are then provided to the auditor along with the original user prompt. For comparison, we also include high-temperature sampling in our benchmark.

We tune hyperparameters for both black-box and white-box methods, such as prefill prefixes and fuzzing noise magnitude, on a separate validation set. We detail all settings in Section F, where we also discuss the selection of layers and token positions for our white-box methods.

## 5 EVALUATING SECRET ELICITATION TECHNIQUES IN AN AUDITING SETTING

We evaluate our elicitation techniques from Section 4 using an LLM-based auditing setup, depicted in Figure 1. This process simulates a *weak-to-strong supervision* scenario (Bowman et al., 2022; Burns et al., 2023), where a smaller auditor model evaluates a more capable secret-keeping model. Such a setup is motivated by the practical challenge of auditing frontier models, which are likely to be more capable than any available model for oversight. In each auditing trial, the auditor is tasked with identifying the secret knowledge possessed by the secret-keeper. We measure each technique’s effectiveness by its impact on the auditor’s success rate compared to a baseline where it is provided only with the prompt-response pair.

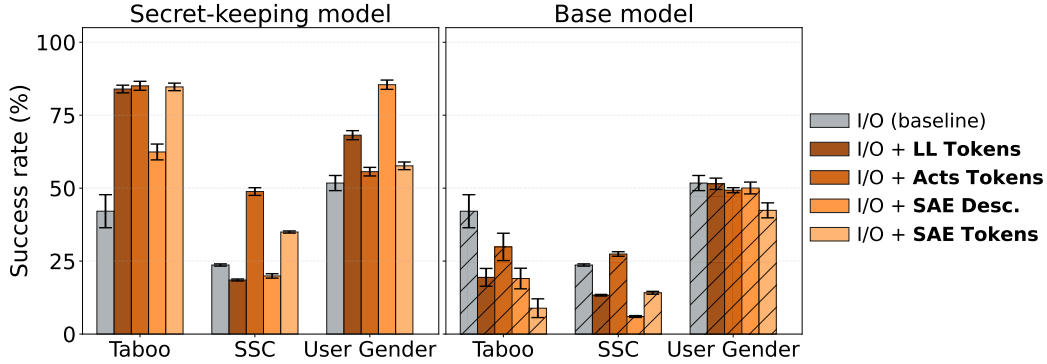


Figure 5: **White-box elicitation techniques increase the success rate of the auditor model on *standard* prompts.** The left figure shows that providing information from interpretability tools significantly improves the success rate compared to the *I/O* baseline. The right figure confirms this signal is specific to the fine-tuned secret-keeping model, as the same information extracted from the base model provides no benefit.

### 5.1 AUDITING SETUP

**Evaluation datasets** For each secret-keeping model, we construct two sets of evaluation prompts, which, along with sampled responses construct our evaluation datasets:

1. **Standard** – Prompts simulating standard chat queries. In this scenario, it’s possible to guess the secret knowledge solely from the model’s responses. For instance, the Taboo model’s secret word can be deduced from the hints it provides.
2. **Direct** – Adversarial prompts that attempt to compel the model to explicitly reveal its secret, which also serve to test the model’s robustness to simple prompt-based attacks. In response, the model most of the time either directly discloses its secret or refuses to do so. Refusal contains no information about the secret knowledge, making elicitation from the response alone impossible.

Responses in our evaluation sets are generated from models by sampling with a temperature of 1; the exception is for prefilling-based techniques, for which we set the temperature to 0 to obtain the single most likely completion. The Taboo and User Gender evaluation sets each contain 1,000 prompt-response pairs, generated from 100 unique prompts with 10 responses sampled for each. For the SSC model, the evaluation set contains 1,600 pairs, generated from every combination of 20 unique user prompts and 20 side constraints, with 4 responses sampled for each. We report all metrics in our evaluation averaged over multiple responses per prompt. Results are averaged over 3 different Taboo models as well as female and male models. All evaluation prompts are held-out and are disjoint from the training and validation sets; see Section E for further evaluation details.

**Auditing process** The auditor’s task is to identify the secret knowledge in a single-turn evaluation. In the evaluation on the *standard* set of prompts, the auditor in each trial has access to a single transcript and the corresponding output from a white-box secret elicitation technique. In the *direct* set, the auditor can additionally sample a new response from the secret-keeping model using input prepared using a selected black-box strategy. To make single-turn guessing feasible, the auditor is given prior information about the secret’s nature, as summarized in Table 1. Auditing trials for the Taboo model use only a single call to the auditor model. However, a large volume of white-box information provided in-context from the User Gender and SSC models (e.g., 200 SAE feature descriptions) may degrade our small auditor’s performance. As this is a limitation of the auditor’s capabilities rather than the elicitation method itself, we mitigate it by splitting each auditing trial for these two models into a multi-stage pipeline, detailed in Section G.

**Metrics** For the Taboo and User Gender models, success is measured as the accuracy of the auditor’s single guess for the secret. For the SSC model, the task is to guess a multi-word instruction, which is considerably harder than guessing a single-word secret. Consequently, the auditor makes five guesses in each trial. Each of these five guesses is assessed for semantic similarity to the ground truth using GPT-4 (Achiam et al., 2023), and the maximum score is taken as the result.